	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 13

Lab Title: **Searhing Algorithms**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective:Binary nd Fibnocci search and their speed to serach element in large data with . . . . . comparison to other search

.

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 13

Tittle : Searching algos

Name : Muammad waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 31 December 24

Objective:

=======(PROBLEM STATEMENT)=====

Binary Search is a searching algorithm that works by repeatedly dividing the search range into two halves.
However, binary search is not efficient in linked lists because linked lists do not support direct access to middle elements. To apply binary search, the linked list must be sorted and the middle node must be found by traversal.
Algorithm (Binary Search in Linked List)
1. Start
2. Create a sorted linked list
3. Set start = head and end = NULL
4. While start is not equal to end
o Find the middle node between start and end
o If middle node data equals key, display position and stop
o If key is less than middle data, set end = middle
o Else set start = middle->next
5. If element is not found, display “Element not found”
6. Stop

CODE

```
1   #include <iostream>
2   using namespace std;
3
4   class Node {
5   public:
6       int data;
7       Node* next;
8       Node(int v) {
9           data = v;
10          next = NULL;
11      }
12  };
```

CODE

```
14  void insert(Node*& head, int val) {
15      Node* n = new Node(val);
16      if (!head || val < head->data) {
17          n->next = head;
18          head = n;
19          return;
20      }
21      Node* t = head;
22      while (t->next && t->next->data < val)
23          t = t->next;
24      n->next = t->next;
25      t->next = n;
26  }
27
28  Node* middle(Node* start, Node* end) {
29      Node* slow = start;
30      Node* fast = start;
31      while (fast != end && fast->next != end) {
32          slow = slow->next;
33          fast = fast->next->next;
34      }
35      return slow;
36  }
```

CODE

```
38  void binarySearch(Node* head, int key) {
39      Node* start = head;
40      Node* end = NULL;
41
42      while (start != end) {
43          Node* mid = middle(start, end);
44          if (!mid) break;
45
46          if (mid->data == key) {
47              cout << "Element found\n";
48              return;
49          }
50          else if (key < mid->data)
51              end = mid;
52          else
53              start = mid->next;
54      }
55      cout << "Element not found\n";
56  }
57
58  void display(Node* head) {
59      cout << "Sorted List: ";
60      while (head) {
61          cout << head->data << " -> ";
62          head = head->next;
63      }
```

CODE

```
67  int main() {
68      Node* head = NULL;
69      int choice, value;
70
71      while (true) {
72          cout << "\n--- BINARY SEARCH GUI ---\n";
73          cout << "1. Insert Node (Sorted)\n";
74          cout << "2. Display List\n";
75          cout << "3. Search Element\n";
76          cout << "4. Exit\n";
77          cout << "Enter choice: ";
78          cin >> choice;
79
80          if (choice == 1) {
81              cout << "Enter value: ";
82              cin >> value;
83              insert(head, value);
84          }
85          else if (choice == 2) {
86              display(head);
87          }
88          else if (choice == 3) {
89              cout << "Enter key: ";
90              cin >> value;
91              binarySearch(head, value);
92          }
```

OUTPUT

```
--- BINARY SEARCH GUI ---
1. Insert Node (Sorted)
2. Display List
3. Search Element
4. Exit
Enter choice: 2
Sorted List: 1 -> 2 -> 4 -> 6 -> 8 -> 10 -> 11 -> 17 -> 21 -> NULL

--- BINARY SEARCH GUI ---
1. Insert Node (Sorted)
2. Display List
3. Search Element
4. Exit
Enter choice: 3
Enter key: 11
Element found
```

=======(PROBLEM STATEMENT)=====

2. Fibonacci Search in Linked List
Objective
To search an element in a sorted linked list using the Fibonacci Search method.
Definition
Fibonacci Search is a searching technique that uses Fibonacci numbers to divide the search space.
In linked lists, Fibonacci Search is inefficient because accessing elements requires sequential traversal, unlike arrays.
Algorithm (Fibonacci Search in Linked List)
1. Start
2. Create a sorted linked list
3. Count the total number of nodes n
4. Generate Fibonacci numbers until the largest Fibonacci number ≥ n
5. Set offset = −1
6. Traverse the linked list to the required Fibonacci index
7. Compare the node data with the key
8. Adjust Fibonacci numbers and offset based on comparison
9. Repeat until the element is found or list ends
10. If not found, display

CODE

```
1   #include <iostream>
2   using namespace std;
3
4   class Node {
5   public:
6       int data;
7       Node* next;
8       Node(int v) {
9           data = v;
10          next = NULL;
11      }
```

CODE

```
14  void insert(Node*& head, int val) {
15      Node* n = new Node(val);
16      if (!head) {
17          head = n;
18          return;
19      }
20      Node* t = head;
21      while (t->next) t = t->next;
22      t->next = n;
23  }
24
25  int count(Node* head) {
26      int c = 0;
27      while (head) {
28          c++;
29          head = head->next;
30      }
31      return c;
32  }
33
34  Node* getNode(Node* head, int index
35      int i = 0;
36      while (head && i < index) {
37          head = head->next;
38          i++;
39      }
```

CODE

```
43  void fibonacciSearch(Node* head, int key) {
44      int n = count(head);
45
46      int f2 = 0, f1 = 1, f = f1 + f2;
47      while (f < n) {
48          f2 = f1;
49          f1 = f;
50          f = f1 + f2;
51      }
52
53      int offset = -1;
54
55      while (f > 1) {
56          int i = min(offset + f2, n - 1);
57          Node* temp = getNode(head, i);
58
59          if (temp->data < key) {
60              f = f1;
61              f1 = f2;
62              f2 = f - f1;
63              offset = i;
64          }
65          else if (temp->data > key) {
66              f = f2;
67              f1 = f1 - f2;
68              f2 = f - f1;
```

CODE

```
        }
        else {
            cout << "Element found at position " << i + 1 << endl;
            return;
        }
    }
    cout << "Element not found\n";
}

void display(Node* head) {
    cout << "List: ";
    while (head) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* head = NULL;
    int choice, value;

    while (true) {
        cout << "\n--- FIBONACCI SEARCH GUI ---\n";
        cout << "1. Insert Node\n";
        cout << "2. Display List\n";
```

CODE

```
94          cout << "2. Display List\n";
95          cout << "3. Search Element\n";
96          cout << "4. Exit\n";
97          cout << "Enter choice: ";
98          cin >> choice;
99
100         if (choice == 1) {
101             cout << "Enter value: ";
102             cin >> value;
103             insert(head, value);
104         }
105         else if (choice == 2) {
106             display(head);
107         }
108         else if (choice == 3) {
109             cout << "Enter key: ";
110             cin >> value;
111             fibonacciSearch(head, value);
112         }
113         else if (choice == 4) {
114             break;
115         }
116         else {
117             cout << "Invalid choice\n";
118         }
119     }
```

OUTPUT

```
--- FIBONACCI SEARCH GUI ---
1. Insert Node
2. Display List
3. Search Element
4. Exit
Enter choice: 2
List: 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 10 -> 15 -> 22 -> 23 -> 32 -> 34 -> NULL

--- FIBONACCI SEARCH GUI ---
1. Insert Node
2. Display List
3. Search Element
4. Exit
Enter choice: 3
Enter key: 22
Element found at position 10
```

3. Linear Search in Linked List

Objective

To search an element in a linked list using Linear Search.

Definition

Linear Search is a simple searching technique in which each node of the linked list is visited sequentially until the required element is found or the end of the list is reached.

This is the most suitable and efficient searching method for linked lists.

Algorithm (Linear Search in Linked List)

1. Start
2. Create a linked list
3. Read the element to be searched (key)
4. Set pointer temp = head and position = 1
5. While temp is not NULL

o If temp->data == key, display position and stop

o Else move temp = temp->next and increment position

6. If end of list is reached, display "Element not found"
7. Stop

CODE

```
1   #include <iostream>
2   using namespace std;
3
4   class Node {
5   public:
6       int data;
7       Node* next;
8       Node(int v) {
9           data = v;
10          next = NULL;
11      }
12  };
```

CODE

```
13  void insert(Node*& head, int val) {
14      Node* n = new Node(val);
15      if (!head) {
16          head = n;
17          return;
18      }
19      Node* t = head;
20      while (t->next) t = t->next;
21      t->next = n;
22  }
23
24  void display(Node* head) {
25      cout << "List: ";
26      while (head) {
27          cout << head->data << " -> ";
28          head = head->next;
29      }
30      cout << "NULL\n";
31  }
32
33  void linearSearch(Node* head, int key) {
34      int pos = 1;
35      while (head) {
36          if (head->data == key) {
37              cout << "Element found at position " << pos << endl;
38              return;
```

CODE

```
        }
        head = head->next;
        pos++;
    }
    cout << "Element not found\n";
}

int main() {
    Node* head = NULL;
    int choice, value;

    while (true) {
        cout << "\n--- LINEAR SEARCH GUI ---\n";
        cout << "1. Insert Node\n";
        cout << "2. Display List\n";
        cout << "3. Search Element\n";
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1) {
            cout << "Enter value: ";
            cin >> value;
            insert(head, value);
        }
        else if (choice == 2) {
```

CODE

```
65              display(head);
66          }
67          else if (choice == 3) {
68              cout << "Enter key to search: ";
69              cin >> value;
70              linearSearch(head, value);
71          }
72          else if (choice == 4) {
73              break;
74          }
75          else {
76              cout << "Invalid choice\n";
77          }
78      }
79      return 0;
80  }
```

OUTPUT

```
--- LINEAR SEARCH GUI ---
1. Insert Node
2. Display List
3. Search Element
4. Exit
Enter choice: 2
List: 1 -> 2 -> 3 -> 4 -> 6 -> 7 -> 8 -> 13 -> 16 -> NULL

--- LINEAR SEARCH GUI ---
1. Insert Node
2. Display List
3. Search Element
4. Exit
Enter choice: 3
Enter key to search: 8
Element found at position 7
```

❖ Conclusion:

- ❖ Linear search is the easiest and most suitable method for searching in a linked list
- ❖ It works by checking each node one by one until the element is found
- ❖ Binary search is not efficient in linked lists because random access is not possible
- ❖ Binary search requires the linked list to be sorted and needs repeated traversal
- ❖ Fibonacci search also needs a sorted linked list and multiple traversals
- ❖ Fibonacci search is better suited for arrays than linked lists
- ❖ Implementing these searches helps in understanding algorithm behavior
- ❖ Choosing the correct search method depends on the data structure used